



Laboratory Report #8

Neural Nets

Name:	Lyka Sophia S. Jardeleza	Date:	06/13/2025
Program:	BSCpE	Section:	A361

A. Instructions:

Part I: Data Loading and Inspection

- 1. Load the dataset using pandas.
- 2. Display the first 10 rows.
- 3. Check for missing values and basic statistics using .info() and .describe().

Part II: Preprocessing

- 1. Separate features (X) and target (y).
- 2. Standardize the features using StandardScaler.
- 3. Split the data into training (80%) and testing (20%) sets.

```
python
CopyEdit
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# Splitting
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Part III: Model Training with MLP Classifier

- 1. Initialize an MLPClassifier with the following:
 - o Hidden layers: (10, 5)
 - o Activation: 'relu'
 - o Solver: 'adam'
 - o Max iterations: 1000

- 2. Train the model on training data.

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(hidden_layer_sizes=(10, 5), max_iter=1000,
activation='relu', random_state=42)
model.fit(X_train, y_train)
```

Part IV: Evaluation

- 1. Predict on the test set.
- 2. Use the following metrics:
 - o accuracy_score
 - o classification_report
 - o confusion_matrix

```
from sklearn.metrics import classification_report, confusion_matrix

y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Part V: Reflection and Analysis

Write short answers to the following:

- 1. What are the most frequent misclassifications?
- 2. How did the activation function impact learning?
- 3. What might happen if we used fewer/more hidden units?
- 4. Could this model be deployed in real maintenance systems? What considerations are needed?
- 5. Visualize the confusion matrix using seaborn.heatmap.
- 6. Try a different activation function (tanh, logistic) and compare results.



Laboratory Report #8

Neural Nets

Screenshot of Output

```
"C:\Users\Lyka Sophia\Documents\DSL\Task8\.venv\Scripts\python.exe" "C:\Users\Lyka Sophia\Documents\DSL\Task8\neuralnets.py"
First 10 rows of the dataset:
  Temperature  Vibration  Runtime  Status
0      79.97      3.98      79.7      1
1      73.62      3.65      95.7      1
2      81.48      3.04      76.2      1
3      90.23      2.55      90.8      2
4      72.66      3.49      43.2      0
5      72.66      3.28     106.4      0
6      90.79      3.63     100.0      2
7      82.67      3.44      75.5      1
8      70.31      3.73     119.8      1
9      80.43      2.63     128.1      1

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Temperature  1000 non-null   float64
1   Vibration    1000 non-null   float64
2   Runtime      1000 non-null   float64
3   Status       1000 non-null   int64
dtypes: float64(3), int64(1)
memory usage: 31.4 KB
None

Dataset Description:
  Temperature  Vibration  Runtime  Status
count  1000.000000  1000.000000  1000.000000  1000.000000
mean     75.193260    3.049650   100.175400    0.753000
std       9.792202    0.698279    29.505332    0.681511
min     42.590000    0.940000    9.400000     0.000000
25%     68.520000    2.577500   80.575000     0.000000
50%     75.255000    3.045000   100.000000    1.000000
75%     81.482500    3.510000   119.850000    1.000000
max    113.530000    5.240000   217.800000    2.000000

Accuracy Score: 0.95

Classification Report:
      precision    recall  f1-score   support

      0       0.98      0.99      0.98        80
      1       0.97      0.93      0.95       100
      2       0.78      0.90      0.84        20

   accuracy          0.95        200
  macro avg       0.91      0.94      0.92        200
weighted avg       0.95      0.95      0.95        200

Confusion Matrix:
[[79  1  0]
 [ 2 93  5]
 [ 0  2 18]]
--- Using Activation Function: tanh ---
Accuracy Score: 0.92

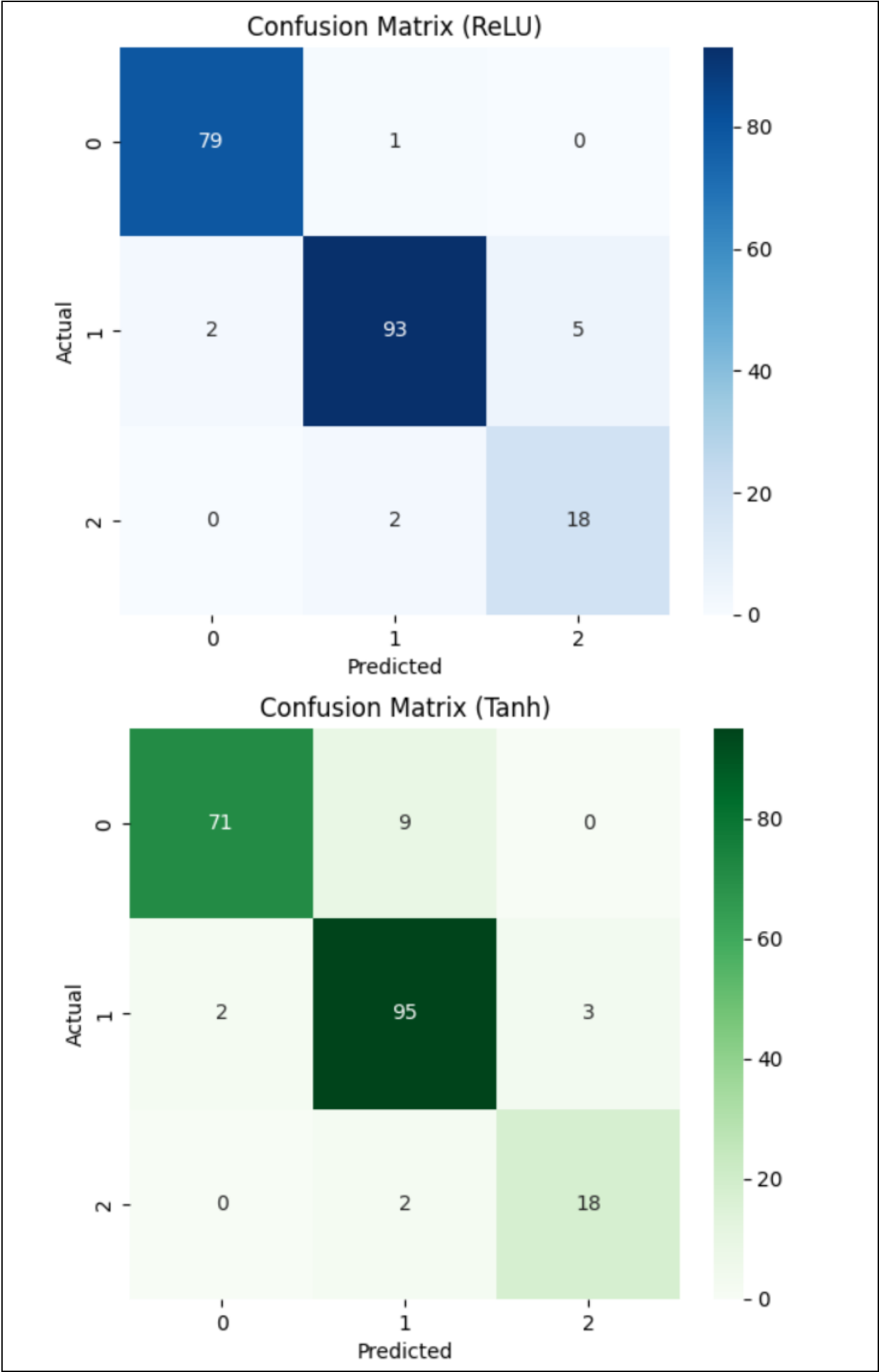
Classification Report:
      precision    recall  f1-score   support

      0       0.97      0.89      0.93        80
      1       0.90      0.95      0.92       100
      2       0.86      0.90      0.88        20

   accuracy          0.92        200
  macro avg       0.91      0.91      0.91        200
weighted avg       0.92      0.92      0.92        200

Confusion Matrix:
[[71  9  0]
 [ 2 95  3]
 [ 0  2 18]]

Process finished with exit code 0
```





Laboratory Report #8

Neural Nets

Source Code

```
# Part I: Data Loading and Inspection
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Load dataset
df = pd.read_csv(r"C:\Users\Lyka Sophia\Documents\DSL\Task8\machine_data.csv")

# Display first 10 rows
print("First 10 rows of the dataset:")
print(df.head(10))

# Check info and missing values
print("\nDataset Info:")
print(df.info())

# Basic statistics
print("\nDataset Description:")
print(df.describe())

# Part II: Preprocessing
# Separate features and target (assuming last column is the target)
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
                                                    test_size=0.2,
                                                    random_state=42)

# Part III: Model Training with MLP Classifier
model = MLPClassifier(hidden_layer_sizes=(10, 5),
                      activation='relu',
                      solver='adam',
                      max_iter=1000,
                      random_state=42)
model.fit(X_train, y_train)

# Part IV: Evaluation
y_pred = model.predict(X_test)

# Metrics
print("\nAccuracy Score:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)

# Visualize Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix (ReLU)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Laboratory Report #8

Neural Nets

```
# Part V: Try a different activation function (tanh)
model_tanh = MLPClassifier(hidden_layer_sizes=(10, 5),
                           activation='tanh',
                           solver='adam',
                           max_iter=1000,
                           random_state=42)

model_tanh.fit(X_train, y_train)
y_pred_tanh = model_tanh.predict(X_test)

print("\n\n--- Using Activation Function: tanh ---")
print("Accuracy Score:", accuracy_score(y_test, y_pred_tanh))
print("\nClassification Report:\n", classification_report(y_test, y_pred_tanh))
print("Confusion Matrix:")
cm_tanh = confusion_matrix(y_test, y_pred_tanh)
print(cm_tanh)

# Visualize confusion matrix for tanh
plt.figure(figsize=(6, 5))
sns.heatmap(cm_tanh, annot=True, fmt='d', cmap='Greens')
plt.title("Confusion Matrix (Tanh)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

C. Findings, Observations, and Comments:

Guiding Questions :

1. What are the most frequent misclassifications?
2. How did the activation function impact learning?
3. What might happen if we used fewer/more hidden units?
4. Could this model be deployed in real maintenance systems? What considerations are needed?
5. Visualize the confusion matrix using seaborn.heatmap.
6. Try a different activation function (tanh, logistic) and compare results.

- The model mostly misclassified Class 0 as Class 1, and sometimes Class 1 as Class 2. This shows that the features, which are the data inputs of these classes may be very similar, making it difficult for the model to tell them apart. When the patterns in the data for different classes are too close, the model can struggle to create accurate separation between them.
- At first, the model used the ReLU activation function, which worked reasonably well because it's fast and effective for many problems. However, changing the activation function to Tanh improved the model's accuracy to 92%. This is likely because Tanh handles both negative and positive values better than ReLU. It helps the model learn more balanced patterns, especially when the input data is centered around zero, making the learning process smoother.
- If the model has too few hidden units, it becomes too simple and might miss important patterns, which are called underfitting. But adding too many hidden units can make it overly complex, causing overfitting, where it performs well on training data but poorly on new, unseen data. Finding the right balance is key.
- This model could be used in real maintenance systems to detect machine problems early. However, before using it in real situations, it must be tested with live data, monitored regularly for performance, and updated as needed. It's also important to ensure the system is safe and dependable, especially if wrong predictions could affect operations.
- Lastly, while Tanh worked well, other functions like Logistic (sigmoid) could also be tested. Still, sigmoid is usually slower and can cause issues in deeper models, so it may not be the best choice in this case.